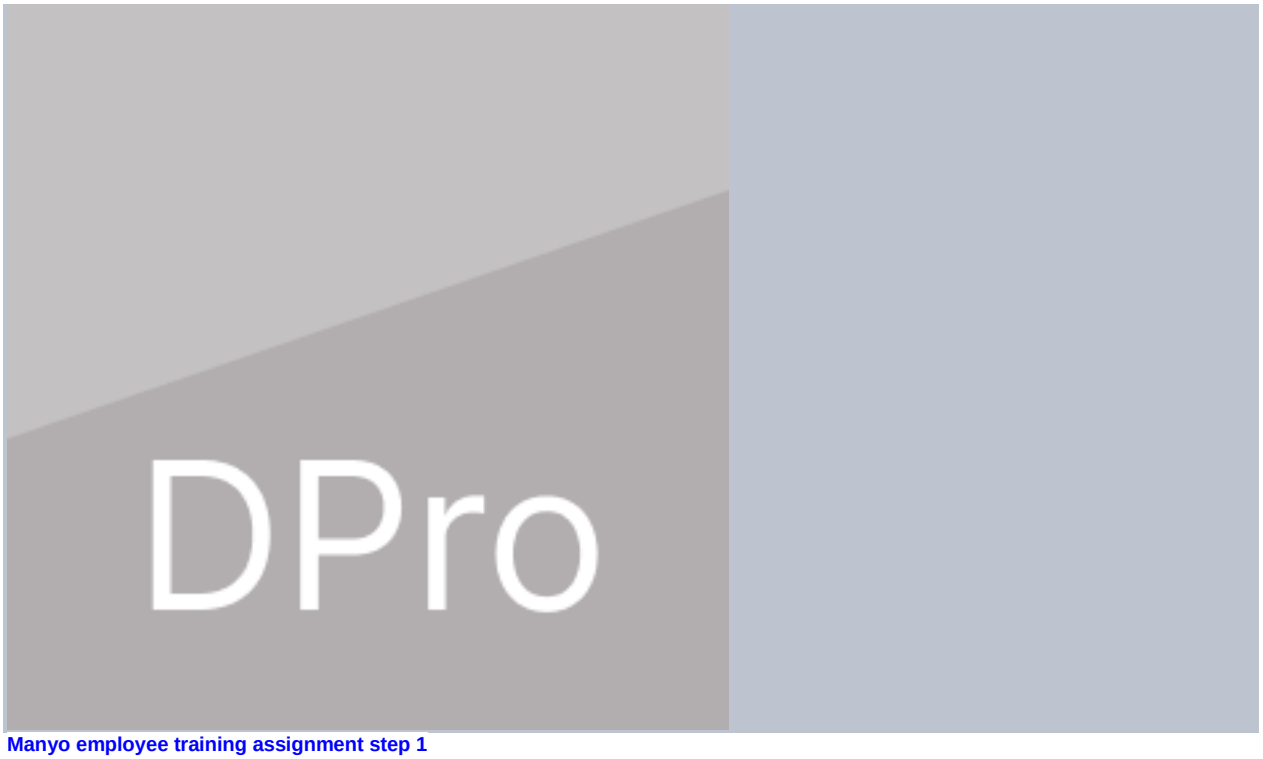


Manyo employee training assignment step 1

- assignment
- Comprehension

Creation Date: 2025-03-28 17:13 Last Update Date: 2025-04-16 17:44 (JST)

Introduction [Everyleaf Corporation New Employee Training Assignment]Manyo employee training assignment step 2

Recommended version for assignments

Please submit your assignment in the following version. You do not need to consider languages not included in this assignment.

Ruby 3.X
Ruby on Rails 7.X

1.Handling of assignment

This assignment is utilized as assignment by DPro with official permission from Manyo Inc. The text has been partially modified to conform to the format of DPro's assignment evaluation.

2.About the license

The copyright is as follows (It is a commercial use contrary to the license, but in our case it can be used as assignment

Original GitHub URL: everyleaf / el-training

3.Goal of assignment

In this assignment, you will develop an application for task management. The functions required for this application can be broadly classified into the following four categories. Also, please write the test code for the implemented function.

1. CRUD function of task
2. User CRUD function
3. Administrator function
4. CRUD function for labels attached to tasks

Think of the image of the application you want to make

- Before proceeding with the design, consider what the finished application will look like. Screen design through paper prototyping is recommended.
    - Also consider how this application will be used (will it be published on the Internet, for internal use, etc…).
- Read the system requirements and consider the data structures you need.
    - What model (table) is needed
    - What information would be needed in the table
- Once you have thought about the data structure, draw it by hand on the model diagram.
  ※ There is no need to create the correct model diagram at this point. Let's make it as an assumption at this point. (If you think it is wrong in future steps, you can modify it.)

4.Important: Flow from development to submission of assignment

This assignment has assignment requirements for each **step and consists of a total of 5 steps** (1 step is treated for each text).

You need to create a new branch for each step and submit your assignment with a pull request raised.
Here, we will explain how to submit an assignment using Step 1 as an example.

1. Create a new `step1` branch from your local master branch and move it
2. Develop in `step1` branch
3. Commit every time you add a feature
4. When you're done with step 1, push to the `step1` branch of the remote repository with the `git push origin step1` command.
5. Receive an evaluation of the assignment
6. After passing, merge the pull request into the `master` branch of the remote repository
7. Locally move from the `step1` branch to the `master` branch
8. Run the `git pull origin master` command to pull the code from the `master` branch of the remote repository into your local `master` branch.
9. Proceed to the next step (text)

After that, create a new branch for each step and submit the assignment in the same procedure.

5.Contents of step 1

In step 1, you will be asked to do the following:

1. Create a repository on GitHub
2. Create a Rails project
3. Implement task CRUD functionality
4. Write a test
5. Deploy to Heroku
6. Link GitHub and Heroku

6.Create a repository on GitHub

First, create a new GitHub repository (repository name is arbitrary).

7.Clone the Rails application for the assignment

Next, prepare a Rails application for the assignment.
Get ready for development by executing the commands written for each of the following steps.

1. Clone and move the Rails application for the assignment

```
$ git clone -b en git@github.com:diveintocode-corp/cdp_web_manyo_task.git
$ cd cdp_web_manyo_task && git checkout -b master && git branch -D en

```

5. Change the registered remote repository to your own remote repository created earlier

```
$ git remote set-url origin "https or ssh of the GitHub repository you created"

```

8. push to the `master` branch

```
$ git push origin master

```

11. `step1` Go to the branch

```
$ git checkout -b step1
```

If the source code is uploaded to the created repository and the local branch is `step1`, you are ready to go.
Run the following command and make sure you can move to the `step1` branch.

```
$ git branch
```

[Execution result]

```
master
* step1
```

8.Create a test

In this assignment, we will create System Spec and Model Spec using R Spec.System Spec is described below.
First,
let ‘s configure the settings for using RSpec.

Introduce RSpec

**

1. RSpec installation and initialization **

Add`gem 'rspec-rails'` `gem 'rexml'` your Gemfile and install it as shown below.

[Gemfile] ``` group :development, :test do # omit gem 'rspec-rails' gem 'rexml' end ```

2. Create an RSpec configuration file

Execute the following command. Then, the files required for RSpec will be generated.

```
$ rails g rspec: install
```

Of the generated files, `spec/spec_helper.rb` describes the overall RSpec settings, and `spec/rails_helper.rb` describes the Rails-specific settings. For now, use the default settings.

Next, in order to display the test results in an easy-to-understand manner, execute the following command and add a setting to convert the test results to document format.

```
$ echo --format documentation >> .rspec
```

9.Configure settings to prevent unnecessary files from being generated.

Configure settings related to the `generator (rails g` command).
Add the following code to `config/application.rb` to prevent unnecessary files from being generated when the `rails g` command is executed.

[config / application.rb]

```
# omit
  config.generators do |g|
    g.assets false
    g.helper false
    g.test_framework :rspec,
      model_specs: true,
      view_specs: false,
      helper_specs: false,
      routing_specs: false,
      controller_specs: false,
      request_specs: false
  end
```

```
# omit
```

With this setting, you can control not to generate unnecessary assets, helpers, test files, etc.when you execute `rails g`

10.Implement CRUD functionality

for tasks

We will implement a CRUD
function to manage tasks.
Here,
let ‘s implement a simple function that allows only “Title” and “contents” of tasks to be registered.
Please proceed with development to satisfy each of the following requirements.

11.Development Requirements

1. The task table name should be `tasks`
2. Create a `tasks`
   table that meets the following requirements

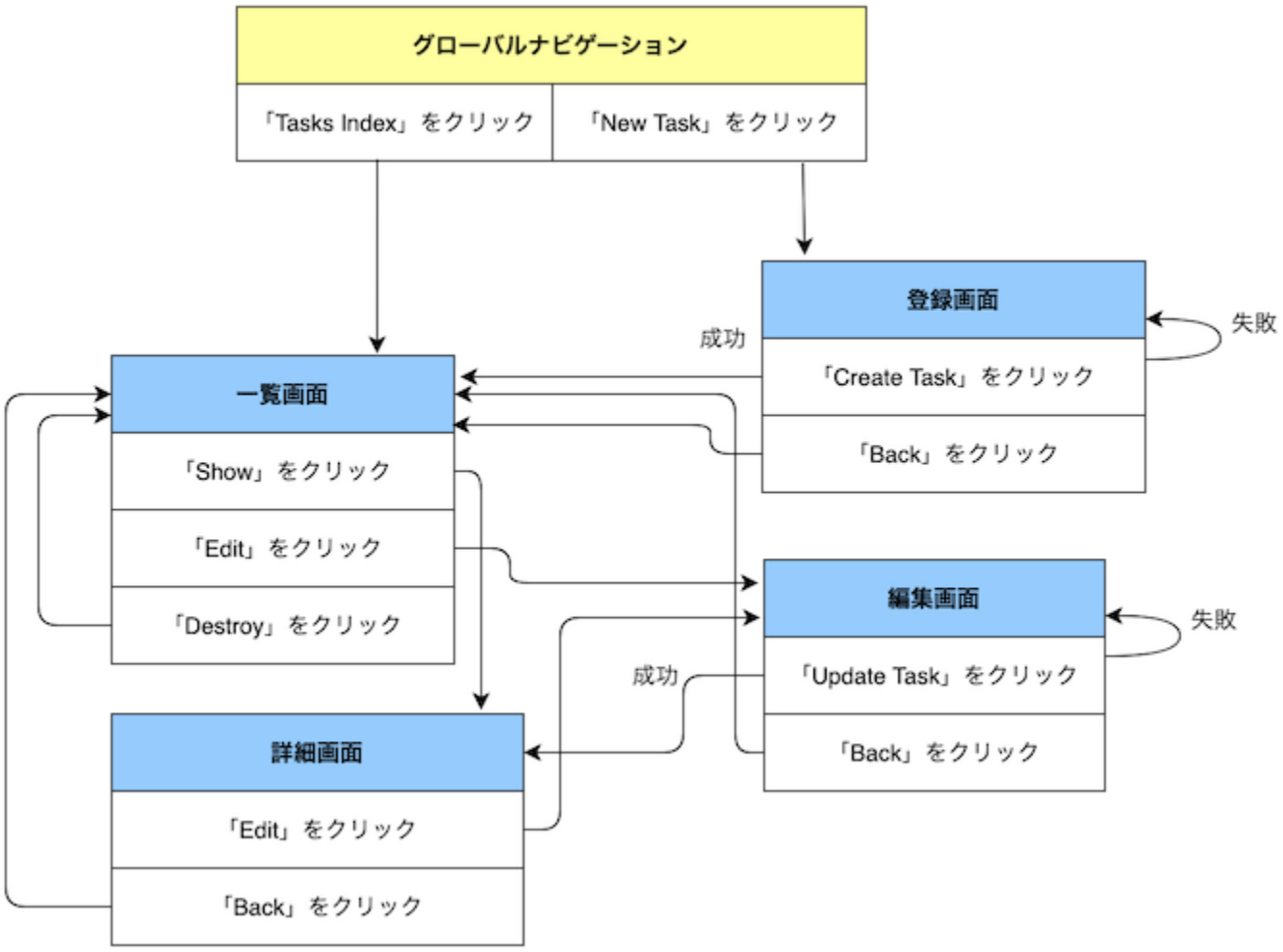
Title	title	string	・Not Null constraint
Contents	content	text	・Not Null constraint

Validation that you set on the database to prevent invalid values from being registered.Validation defined in the model, Validation in the database has the effect of increasing the security of security.
By creating a migration file like the one below and performing the migration, you can set Validation in the database so that empty values are not saved in the `title`

```
class ChangeColumnNullTasks < ActiveRecord::Migration
  change_column :tasks, :title, :string, null: false
end
```

12.Screen Transition Requirements

1. Make a transition according to the screen transition diagram below



1. The path prefix is generated as follows

root	Task list screen
tasks	Task list screen
new\ _task	Task registration screen
task	Task details screen

edit\ _task Task edit screen

2. You can check the path prefix with the `rails routes`

3.

13.Screen Design Requirements

> Internationalization of design and characters will be dealt with in another step.Here,
> let 's implement only the functions without applying the design as shown below.

Global navigation

> [![Image from Gyazo](https://t.gyazo.com/teams/diveintocode/6fd543ba31348ee2d71c554cf8b67194.png)](https://diveintocode.gyazo.com/6fd543ba31348ee2d71c554cf8b67194)
>
> *
> With reference to the above, create a global navigation that meets the following requirements
>
> |
> Target | Character to be displayed | HTML id attribute given to the target |
> |
> -- - | -- - | -- - |
> |
> Link to task list screen | Tasks Index | tasks - index |
> |
> Link to task registration screen | New Task | new - task |

Task list screen

> [![Image from Gyazo](https://t.gyazo.com/teams/diveintocode/d3c396815b6c955f24a1f1e93b63da40.png)](https://diveintocode.gyazo.com/d3c396815b6c955f24a1f1e93b63da40)

1. Create a task list screen that meets the following requirements by referring to the above.

|
Target | Character to be displayed | HTML class attribute given to the target |
|
-- - | -- - | -- - |
|
Screen Title | Tasks Index Page | |
|
Table header | Title, Content, Created¥ _at | |
|
Link to task details screen | Show | show - task |
|
Link to task edit screen | Edit | edit - task |
|
Link to delete a task | Destroy | destroy - task |
2. Display the task Title and content, creation date and time in a list

Task registration screen

[![Image from Gyazo](https://t.gyazo.com/teams/diveintocode/228715d5cc790ac847ec6a98ca4b3ca2.png)](https://diveintocode.gyazo.com/228715d5cc790ac847ec6a98ca4b3ca2)

*
Create a task registration screen that meets the following requirements by referring to the above.

|
Target | Character to be displayed | HTML id attribute given to the target |
|
-- - | -- - | -- - |
|
Screen Title | New Task Page | |
|
Title form label | Title Title | |
|
Content form label | Content | |

|
Button to register a task | Create Task | create - task |
|
Link to
return to the task list screen | Back | back |

Task details screen

[![Image from Gyazo](https://t.gyazo.com/teams/diveintocode/09eecd3bbf8a39144768f96ab02eca9e.png)](https://diveintocode.gyazo.com/09eecd3bbf8a39144768f96ab02eca9e)

1. Refer to the above and create a user list screen that meets the following requirements.

|
Target | Character to be displayed | HTML id attribute given to the target |
|
-- - | -- - | -- - |
|
Screen Title | Show Task Page | |
|
Title item | Title Title | |
|
Content item | Content | |
|
Creation date and time item | Created¥ _at | |
|
Link to task edit screen | Edit | edit - task |
|
Link to
return to the task list screen | Back | back |
2. Display the task Title and content, creation date and time

Task edit screen

[![Image from Gyazo](https://t.gyazo.com/teams/diveintocode/083c746cd690553021858c5b6900ed95.png)](https://diveintocode.gyazo.com/083c746cd690553021858c5b6900ed95)

*

Create a user edit screen that meets the following requirements by referring to the above.

|
Target | Character to be displayed | HTML id attribute given to the target |
|
-- - | -- - | -- - |
|
Screen Title | Edit Task Page | |
|
Title form label | Title Title | |
|
Content form label | Content | |
|
Button to register a task | Update Task | update - task |
|
Link to
return to the task list screen | Back | back |

14.Functional Requirements

1. Validation fails when registering or editing a task, display the Validation message according to the following conditions.

|
Conditions | Validation message |
|
-- - | -- - |
|
Title is entered | Title can 't be blank | |
If no content is entered | Content can 't be blank |
2. Display the flash message according to the following conditions

|
Conditions | Flash message to display |
|

-- - | -- - |
|
If the task registration is successful | Task was successfully created. |
|
If the task update is successful | Task was successfully updated. |
|
If you delete a task | Task was successfully destroyed. |
3. Display the text "Are you sure?"
In the confirmation dialog when you click the link to delete the task.

For Vagrant

When using RSpec with Vagrant, you need to install `chromedriver`
and `google-chrome（headless）`
If you can confirm the version by executing the following command, installation is not necessary. ``` $ chromedriver --version ChromeDriver 79.0.3945.16 (93fcc21110c10dbbd49bbff8f472335360e31d05-refs/branch-heads/3945@{#262}) $ google-chrome --version Google Chrome 79.0.3945.88 ```

If you don ‘t see the version, run the following command to install `chromedriver` and `google-chrome（headless）`

1. Install `google-chrome（headless）`

```
$ curl -sS -o - https://dl-ssl.google.com/linux/linux_signing_key.pub | sudo apt-key add
$ sudo sh -c 'echo "deb [arch=amd64] http://dl.google.com/linux/chrome/ deb/ stable main" >> /etc/apt/sources.list.d/google-chrome.list'
$ sudo apt-get -y update
$ sudo apt-get -y install google-chrome-stable
```

1. Install `chromedriver`

```
$ sudo apt-get install -y unzip
$ wget https://chromedriver.storage.googleapis.com/2.41/ chromedriver_linux64.zip
$ unzip chromedriver_linux64.zip
$ sudo mv chromedriver /usr/bin/chromedriver
$ sudo chown root:root /usr/bin/chromedriver
$ sudo chmod +x /usr/bin/chromedriver
```

1. `spec_helper.rb`
   the following code to `headless_chrome`
   and set the `no-sandbox`

[spec\ _helper.rb]

```
RSpec.configure do |config|
 config.before(:each) do |example|
  if example.metadata[:type] == :system
   driven_by :selenium, using: :headless_chrome, screen_size: [1400, 1400]  do |options|
    options.add_argument('no-sandbox')
   end
  end
 end
 # omit
end
```

If you can ‘t install with the above command, check how [to set up Selenium with ChromeDriver on Ubuntu 18.04 & 16.04.]

15.Create a Model Spec

From here, we will write the Model Spec.The items to be tested are as follows.

1. Validation fails
   if the task Title
2. Validation fails
   if task description is empty
3. If the task Title and description have values, they will be registered.

First,
let ‘s create a file for Model Spec. By executing the following command,
the `spec/models/task_spec.rb` file for Model Spec and the `spec/factories/tasks.rb` file for FactoryBot will be generated (FactoryBot will be explained later).

```
$ rails g rspec:model task
```

The basic format of Model Spec has the following structure.
`describe` in “or test of how specifications (function)”, `context`
“test the contents of classified situation, the state”
is in , `it`
are in you describe the “operation to expect.”

[spec / models / task\ _spec.rb]

```
require 'rails_helper'

RSpec.describe 'Task model function', type: :model do
  describe 'Validation test' do
    context 'If the task Title is an empty string' do
      it 'Validation fails' do
      end
    end

    context 'If the task description is empty' do
      it 'Validation fails' do
      end
    end

    context 'If the task Title and description have values' do
      it 'You can register a task' do
      end
    end
  end
end
```

Delete the contents described by
default in the `spec/models/task_spec.rb`
At this point,
let ‘s check if Model Spec works properly. To execute the Model Spec described in `spec/models/task_spec.rb`

```
$ bundle exec rspec spec/models/task_spec.rb
```

If the following screen is displayed, there is no problem.

```
[manyo (step1 *)]$ bundle exec rspec spec/models/task_sp

Task Model Function
    Validation Testing
        If the task title is empty
            Validation error occurred
        If the task details are empty
            Validation error occurred
        If the task title and details contain values
            Tasks can be registered

Finished in 0.20086 seconds (files took 9.46 seconds to
3 examples, 0 failures
```

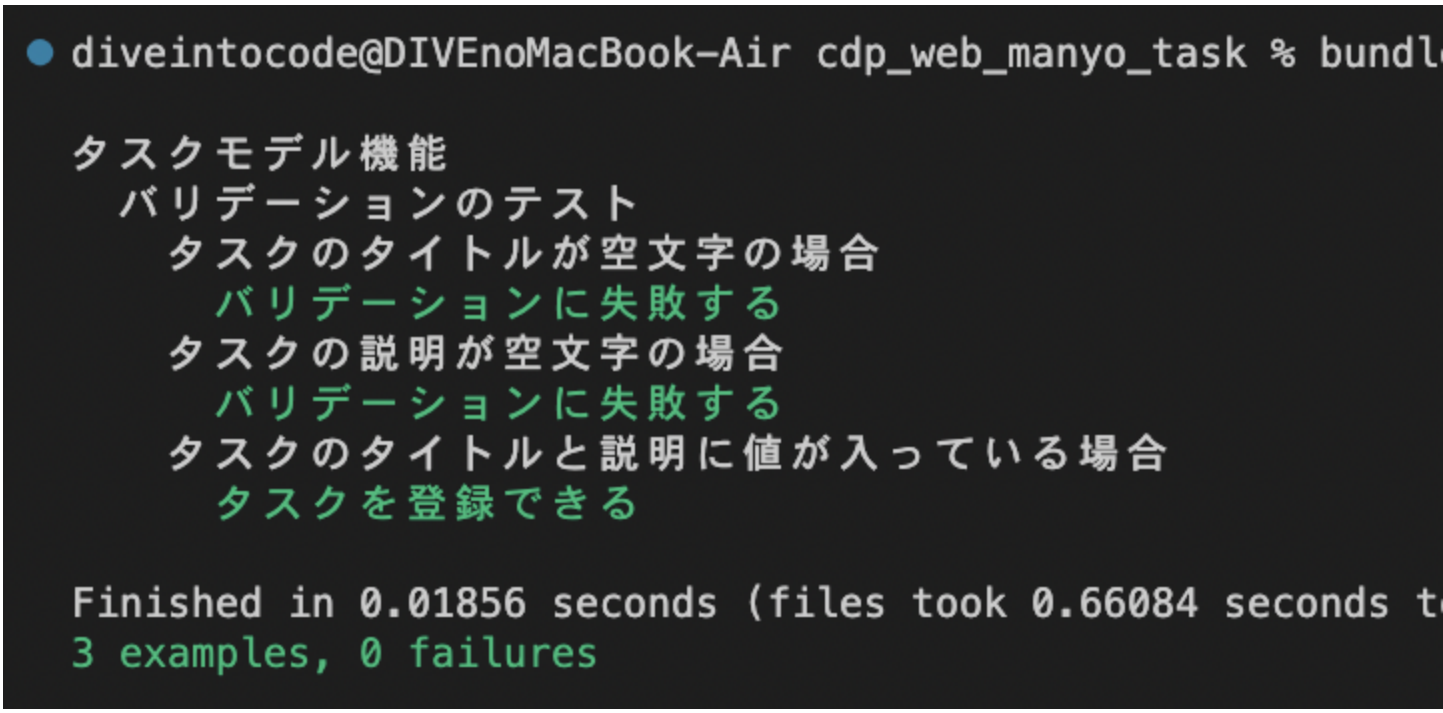
Next, we will write tests for each item in the template.
Here is a sample code only for the items “If Title of the task is an empty string” and “Validation fails” in the “Test for Validation.

[spec / models / task_spec.rb]

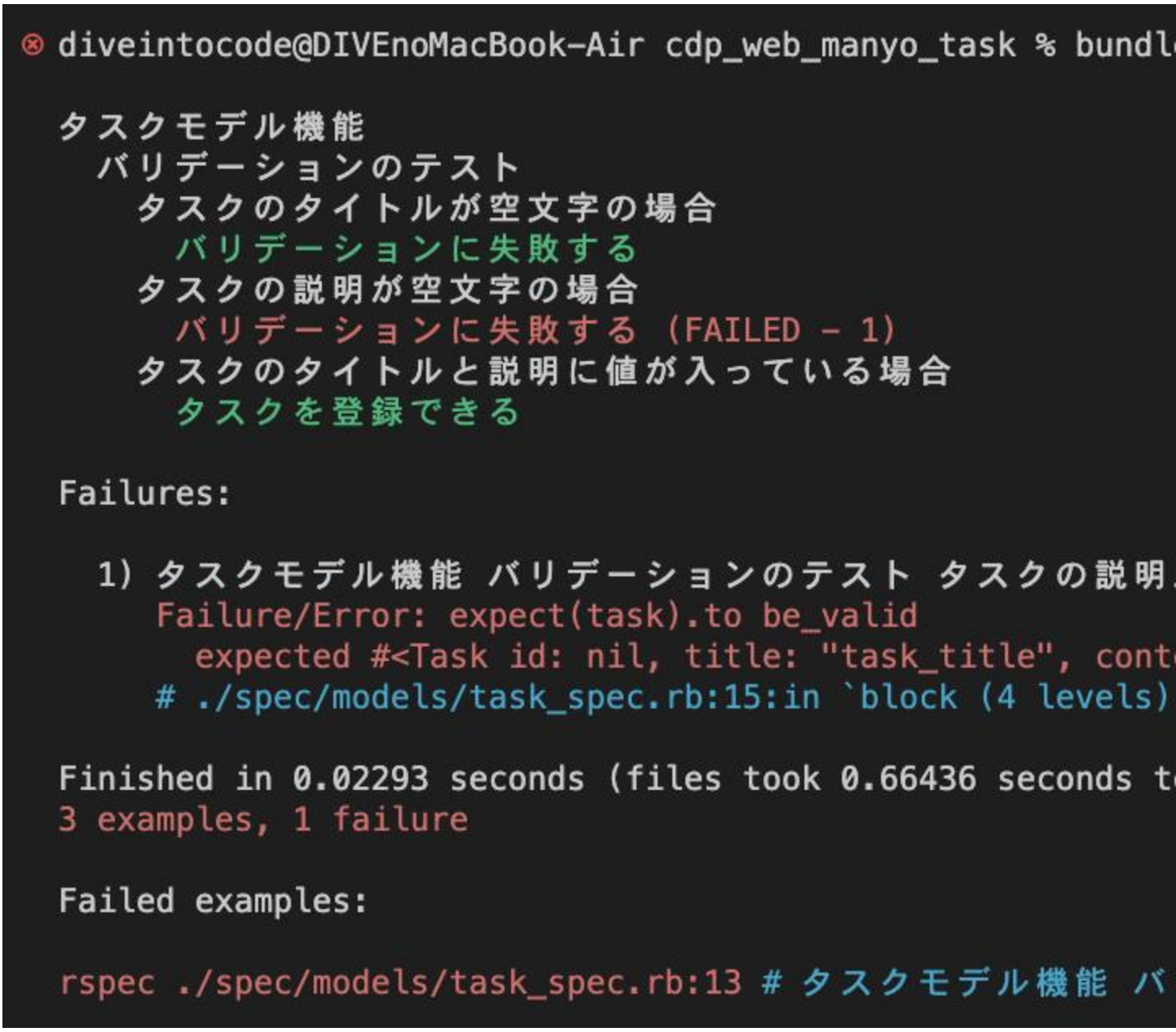
```
# omit
context 'If the task Title is an empty string' do
  it 'Validation fails' do
    task = Task.create(title: '', content: 'Create a proposal.')
    expect(task).not_to be_valid
  end
end
# omit
```

With this as a hint,
let ‘s create the remaining two tests while examining them by yourself, and let’ s make all the tests successful.

If all the tests you create are successful, you will see the following results:



On the other hand,
if one of the tests fails, the following results are displayed



If the tests fail, modify the Model Spec so that all tests succeed.

Model Spec requirements

1. Successful "Validation fails if task Title is empty string"
test 2. Successful test
for "Validation fails if task description is empty"
3. Title and description have values, you can register the task "
4. There is a test written that can verify that the code is correct

16.Create a System Spec

From here, we will write the System Spec.System Spec is a test
function
for executing E2E(end - to - end) tests.The E2E test is a test to confirm that the "from start to finish (from the time the user comes to the page to the time when the user fulfills the purpose and leaves)" of the website or application works as expected.¥*In Rails applications, this is a test that confirms the process from opening the browser to closing it(using the
function and returning the result of use), and this test is collectively called System Spec.The test code itself is written in a directory called `spec/system`
Until Ver3 .7 of RSpec, it was called Feature Spec.

Now
let 's actually write the System Spec. The items to be tested are as follows.

1. When you register a task, the registered task is displayed.2. A list of registered tasks is displayed on the list screen.3. When you move to any task details screen, the contents of that task are displayed.

The basic format of System Spec is similar to that of Model Spec.

[spec / system / tasks¥ _spec.rb]

```
require 'rails_helper'

RSpec.describe 'Task management function', type: :system do
  describe 'Registration function' do
    context 'When registering a task' do
      it 'The registered task is displayed' do
      end
    end
  end

  describe 'List display function' do
    context 'When transitioning to the list screen' do
      it 'A list of registered tasks is displayed' do
      end
    end
  end
```

```
  describe 'Detailed display function' do
    context 'When transitioned to any task details screen' do
      it 'The content of the task is displayed' do
      end
    end
  end
end
```

First, create a directory called `system`
`spec`
directory, and create a file called `tasks_spec.rb`
Once created, paste the above code into this file.

At this point,
let ‘s check if System Spec works properly. To run the System Spec listed in `spec/system/tasks_spec.rb`

```
$ bundle exec rspec spec/system/tasks_spec.rb
```

If the following screen is displayed, there is no problem.

```
[manyo_test_app (step1 *)]$ bundle exec rspec spec

タスク管理機能
  登録機能
    タスクを登録した場合
      登録したタスクが表示される
  一覧表示機能
    一覧画面に遷移した場合
      登録済みのタスク一覧が表示される
  詳細表示機能
    任意のタスク詳細画面に遷移した場合
      そのタスクの内容が表示される

Finished in 0.82591 seconds (files took 3.51 secon
3 examples, 0 failures
```

Next, we will write tests corresponding to each item of the template.Here, we will introduce the code that serves as a sample only
for the items of "list display function", "when transitioning to the list screen", and "displaying the registered task list".

[spec / system / tasks¥ _spec.rb]

```
 # omit
 describe 'List display function' do
   context 'When transitioning to the list screen' do
     it 'A list of registered tasks is displayed' do
       # Register a task for use in the test
       Task.create!(title: 'Document preparation', content: 'Create a proposal.')
       # Transition to task list screen
       visit tasks_path
       # Expect (confirm / expect) that the character string "document creation" is included in the visited page (in this case, the task list screen).
       expect(page).to have_content 'Document preparation'
```

```
      # If the result of expect is "true", the test result is output as success, and if the result of expect is "false", the test result is output as failure.
    end
  end
end
# omit
```

`have_content`
such as have\ _content used in `expect`
that compares the actual value with the expected value and determines if they match is called a matcher.RSpec comes with a variety of matchers, so check them out as needed.

With this as a hint,
let ‘s create the remaining two tests while examining them by yourself, and let’ s make all the tests successful.

**
If you are using Vagrant and get an error like the one below, check again to see if `chromedriver`
and `google-chrome (headless)`
are installed. **

1) Task management function Task list screen When creating a task The created task is displayed
Got 0 failures and 2 other errors:

1.1) Failure/Error: visit tasks_path

```
Selenium::WebDriver::Error::UnknownError:
  unknown error: Chrome failed to start: exited abnormally
    (unknown error: DevToolsActivePort file doesn't exist)
    (The process started from chrome location /usr/bin/google-chrome is no longer running, so ChromeDriver is assuming that Chrome has crashed.)
```

17.Create test data using FactoryBot

When testing with RSpec, it is common to create test data using a feature called `FactoryBot` A template for test data generated using `FactoryBot`
By default, test data is saved in a database dedicated to testing, and it is automatically saved and deleted for each test. Therefore, test data does not accumulate.

To use FactoryBot, you need to install `gem 'factory_bot_rails'`
`gem 'factory_bot_rails'` to your Gemfile and install it as shown below.

[Gemfile]

```
group :development, :test do
  # omit
  gem 'factory_bot_rails'
end
```

Now
let ‘s use FactoryBot to define a factory for the task.
Create a directory called `factories`
under the `spec`
directory, and create a file called `tasks.rb`
Paste the following code into the created `spec/factories/tasks.rb`

[spec / factories / tasks.rb]

```
# The statement "Use FactoryBot"
FactoryBot.define do
  # Name the test data to be created "task"
  # 「task」If the test data name is a snake case of an existing class name, such as
  factory :task do
    title { 'Document preparation' }
    content { 'Create a proposal.' }
  end
  # Name the test data to be created "second_task"
  # 「second_task」If you want to use a nonexistent class name snake case as the test data name,
as in the following example,`class` option to specify which class of test data to create
  factory :second_task, class: Task do
    title { 'send e-mail' }
    content { 'Send a sales email to a customer.' }
  end
end
```

The above is the code to create test data for a task using FactoryBot.
`spec/factories/tasks.rb`, you can use test data using FactoryBot.
Let’s rewrite the test code written in `spec/system/tasks_spec.rb` earlier to the code using FactoryBot.

[spec / system / tasks_spec.rb]

```
  # omit
  describe 'List display function' do
    context 'When transitioning to the list screen' do
      it 'A list of registered tasks is displayed' do
        # Register a task for use in the test
-       Task.create!(title: 'Document preparation', content: 'Create a proposal.')
+       FactoryBot.create(:task)
        # Transition to task list screen
        visit tasks_path
        # Expect (confirm / expect) that the character string "task_title" is included in the visited page (in this case, the task list screen).
        expect(page).to have_content 'Document preparation'
        # If the result of expect is "true", the test result is output as success, and if the result of expect is "false", the test result is output as failure.
      end
    end
  end
  # omit
```

In this way, FactoryBot allows you to create test data with simpler code.
It is also possible to override the factory properties when generating test data. In that case, write as follows.

```
# Generate test data while overwriting the property title declared at the time of factory definition
FactoryBot.create(:task, title: 'Creating meeting materials')
# Generate test data while overwriting the properties title and content declared at the time of factory definition
FactoryBot.create(:second_task, title: 'Competitive investigation', content: 'Investigate the services of other companies.')
```

** Based on the explanation so far,
let ‘s write the test code for the remaining two items by yourself so as to meet the following requirements.**

System Spec requirements

1. Creating test data using FactoryBot
2. Successful test of “If you register a task, the registered task will be displayed”
3. Successful test of “When transitioning to the list screen, the registered task list is displayed”

4. Successful test of “When transitioning to any task details screen, the contents of that task are displayed”
5. There is a test written that can verify that the code is correct

Also, since the following explains whether the test code is written correctly and how to debug the test,
let ‘s check this as well.

18.Verify

if the test fails

In order to verify that the test itself is written correctly, it is effective to intentionally set the wrong result as the expected value and check the test result.Rewrite the test code as follows and
try running the test again.

[spec / system / tasks\ _spec.rb]

```
# omit
describe 'List display function' do
  context 'When transitioning to the list screen' do
    it 'The created task list is displayed' do
      FactoryBot.create(:task)
      visit tasks_path
      # Deliberately set the wrong result as the expected value
      expect(page).to have_content 'Create a budget for the project.'
    end
  end
end
# omit
```

As a result, the result that the test failed is returned as shown below, and it can be proved that the test fails if the wrong result is set as the expected value. Conversely, if the test succeeds even if you set the wrong result as the expected value, you can determine that the test itself is problematic.

```
Task Management Functions
  Registration Functions
    When you register a task
      Registered tasks are displayed
  List function
    When you move to the list screen
Capybara starting Puma...
* Version 3.12.6 , codename: Llamas in Pajamas
* Min threads: 0, max threads: 4
* Listening on tcp://127.0.0.1:49859
      The list of created tasks is displayed(FAILED
  Detailed display function
    When transitioning to any task detail screen
      The contents of that task are displayed

Failures:

  1) Task Management Functions List function W
     Failure/Error: expect(page).to have_content '
       expected to find text "second_task_title" i

     [Screenshot]: tmp/screenshots/failures_r_spec


     # ./spec/system/task_spec.rb:15:in `block (4

Finished in 6.07 seconds (files took 2.36 seconds
3 examples, 1 failure

Failed examples:

rspec ./spec/system/task_spec.rb:11 # Task Manage
```

It's very important to make sure that the test itself is correct, so be sure to write a new test and make sure it fails at least once.

System Spec also saves a screenshot of the screen that failed automatically when the test failed. You can see the saved `tmp/screenshots`

19.System Spec debugging

`binding.irb`, the following verification can be performed.

- Is the test data registered correctly?
- Is it possible to transition to the target page?
- Is the data displayed on the page appropriate?

You can debug while displaying the browser by adding the debug tool to the part you want to verify as shown below and executing the test.

[spec / system / tasks_spec.rb]

```
# omit
describe 'List display function' do
  context 'When transitioning to the list screen' do
    it 'A list of registered tasks is displayed' do
      FactoryBot.create(:task)
      visit tasks_path
      # Debug immediately after page transition
      binding.irb
      expect(page).to have_content 'Create a budget for the project.'
    end
  end
end
# omit
```

20.Deploy to Heroku

Deploy your application to Heroku using the following instructions.
If you get an error when creating a new Heroku account, please try creating an account using an account other than gmail.

1. Make sure all your work has been committed

2. Create a new application on Heroku
3. Deploy to Heroku

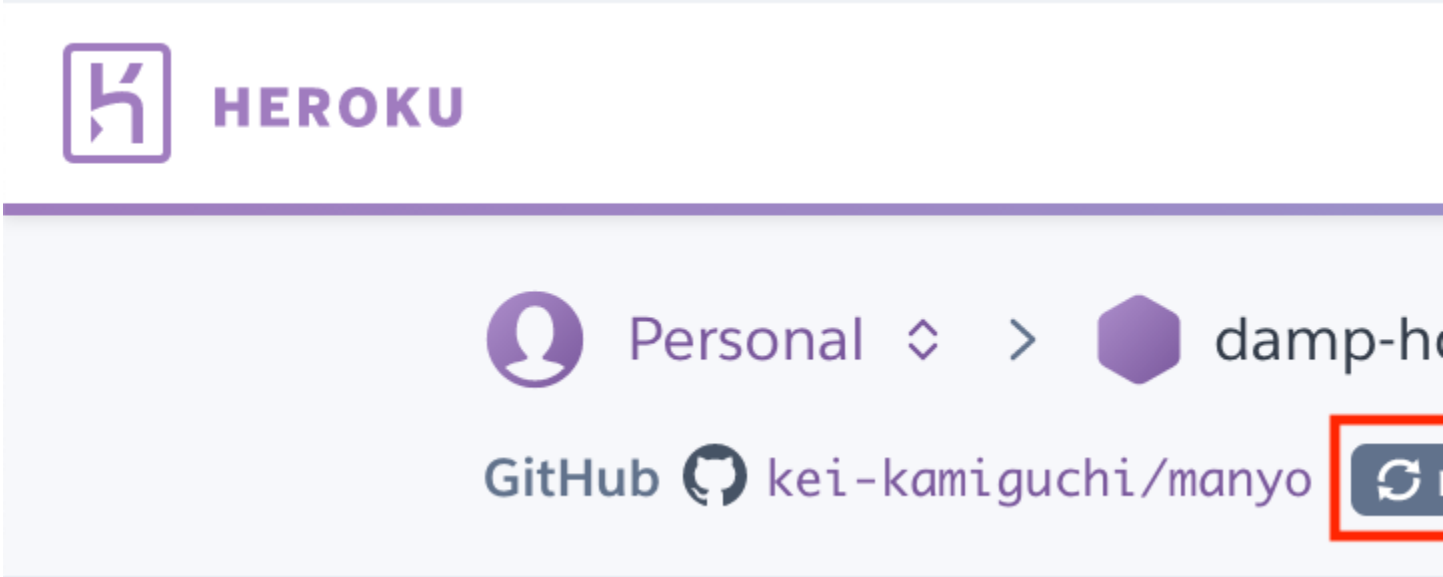
To deploy the application developed in the `step1` `master` branch for step 3, run the following command:

```
$ git push heroku step1:master
```

If you would like a refresher on how to deploy to Heroku, please refer to the Deploying to Heroku series.
*** Also, since anyone can refer to the application deployed on Heroku, please do not post information that is bad for publishing.**

21.Link GitHub and Heroku

GitHub and Heroku are automatically pushed to Heroku once the code is merged into the GitHub master (pull requests are merged) so that you don't have to manually deploy to Heroku every time you develop. Please link (For how to link, refer to the articles on the Web and set it while checking by yourself).
When linked, the following display will appear. Please take a screenshot to show this part and attach the image to the comment section of the assignment



22.Pass requirements

Basic requirements

1. assignment has been submitted with the pull request raised (not merged)
2. Ruby version must be 3.3.0
3. Rails version must be 6.1 series.

4. The database uses PostgreSQL
5. Make settings so that unnecessary files are not generated
6. The task table name should be `tasks`
7. `tasks` table is created according to the requirements
8. Screenshots of GitHub and Heroku working together have been submitted
9. Deploy to Heroku's master branch and reflect the content of step 1

Screen design requirements

1. HTML id attribute and
   class attribute must be assigned according to the requirements
2. Display characters, links, and buttons on each screen as required
3. Display the task Title, description, and creation date and time in a list on the list screen.
4. Display the task Title and content, creation date and time on the details screen

Screen transition requirements

1. Transition according to the screen transition diagram
2. The path prefix is generated as follows
Prefix	Access destination
—	—
root	Task list screen
tasks	Task list screen
new_task	Task registration screen
task	Task details screen
edit_task	Task edit screen

Functional requirements

1. Display the text "Are you sure?" In the confirmation dialog when you click the link to delete the task.
2. Validation fails when registering or editing a task, display the Validation message according to the following conditions.
Conditions	Validation message
—	—
Title is entered	Title can't be blank
If no content is entered	Content can't be blank
3. Display the flash message according to the following conditions
Conditions	Flash message to display
—	—
If the task registration is successful	Task was successfully created.
If the task update is successful	Task was successfully updated.
If you delete a task	Task was successfully destroyed.

Test requirements

- **Model Spec**
    1. Successful “Validation fails if task Title is empty string” test
    2. Successful test for “Validation fails if task description is empty”
    3. Title and description have values, you can register the task”
    4. There is a test written that can verify that the code is correct
- **System Spec**
    1. Creating test data using FactoryBot
    2. Successful test of “If you register a task, the registered task will be displayed”
    3. Successful test of “When transitioning to the list screen, the registered task list is displayed”
    4. Successful test of “When transitioning to any task details screen, the contents of that task are displayed”
    5. There is a test written that can verify that the code is correct

23.Receive an evaluation of your assignment.

The evaluation of assignment is divided into two stages: “getting **an automatic evaluation”** and **“getting a mentor’s evaluation”**.

1. Receive automatic evaluation

Once you’ve met all your requirements, push from the `step1` branch you’re currently working on the `step1` branch in the remote repository.
To push from the local `step1` branch to the `step1` branch of the remote repository, run the following command:

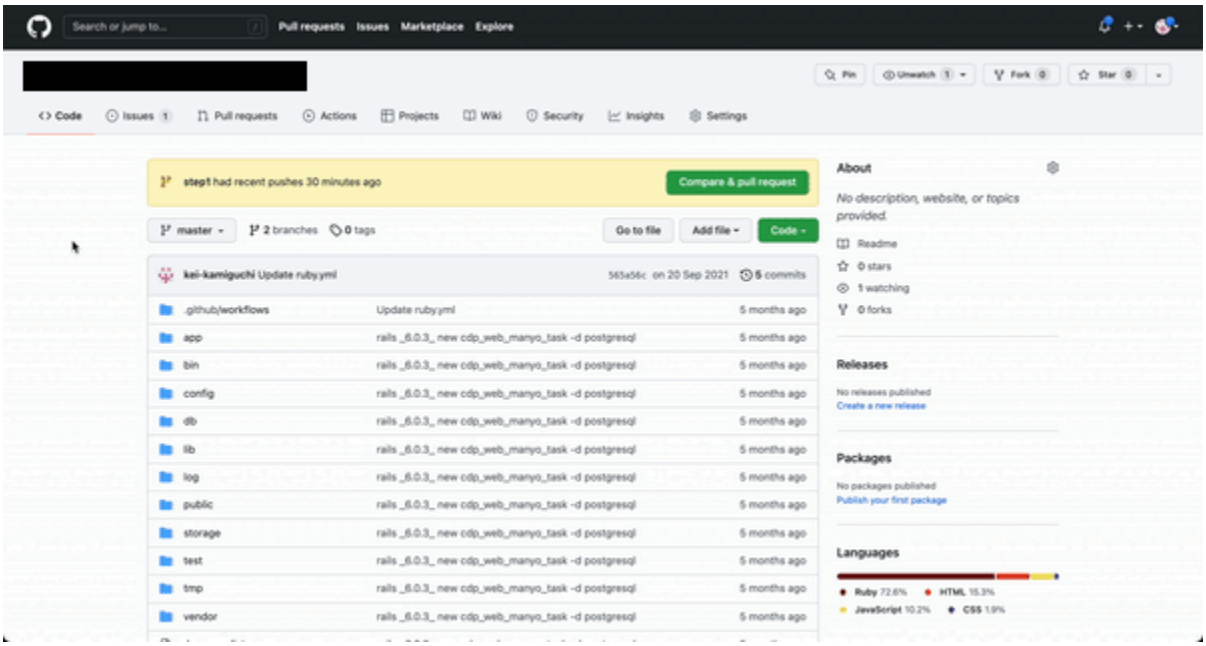
```
$ git push origin step1
```

Then, the evaluation will be performed automatically, and the following evaluation results will be sent by e-mail in about 5 to 10 minutes.

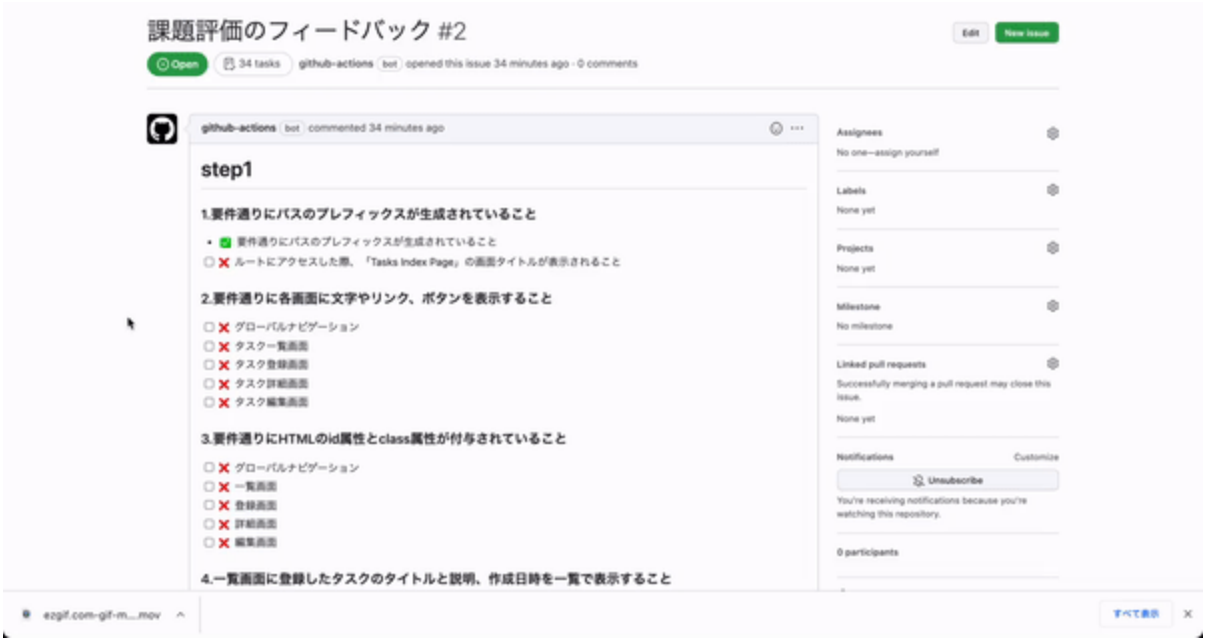
step1

1. The path prefix is generated according to the requirements
   - ✅ The path prefix is generated according to the requirements
   - ❌ When accessing the X route the screen title of "Task Index page" is display
2. Display characters, links and buttons on each screen as required
   - ❌ Global Navigation
   - ❌ Task list screen
   - ❌ Task registration screen
   - ❌ Task detail screen
   - ❌ Task edit screen

Also, an issue with the same content as the result notified by email will be created in your remote repository.
Let’s open the issue as follows.



Items marked with ✕ are the parts that need to be corrected. Let’s fix the code in that part.
Since each item of the issue is in the form of a check box, you can manage where you have corrected it by checking the corrected part.

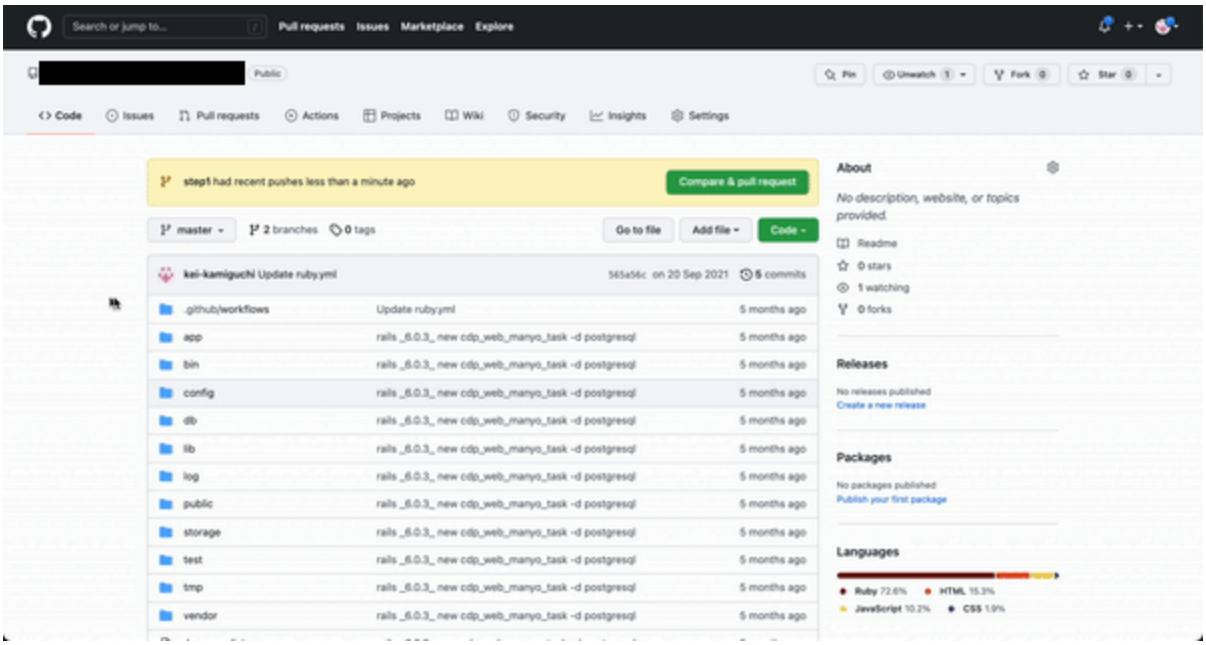


When all the items have been modified, push again to the `step1` branch.
Repeat the above corrections until all items are marked with ☑.
You can receive automatic evaluation as many times as you like.

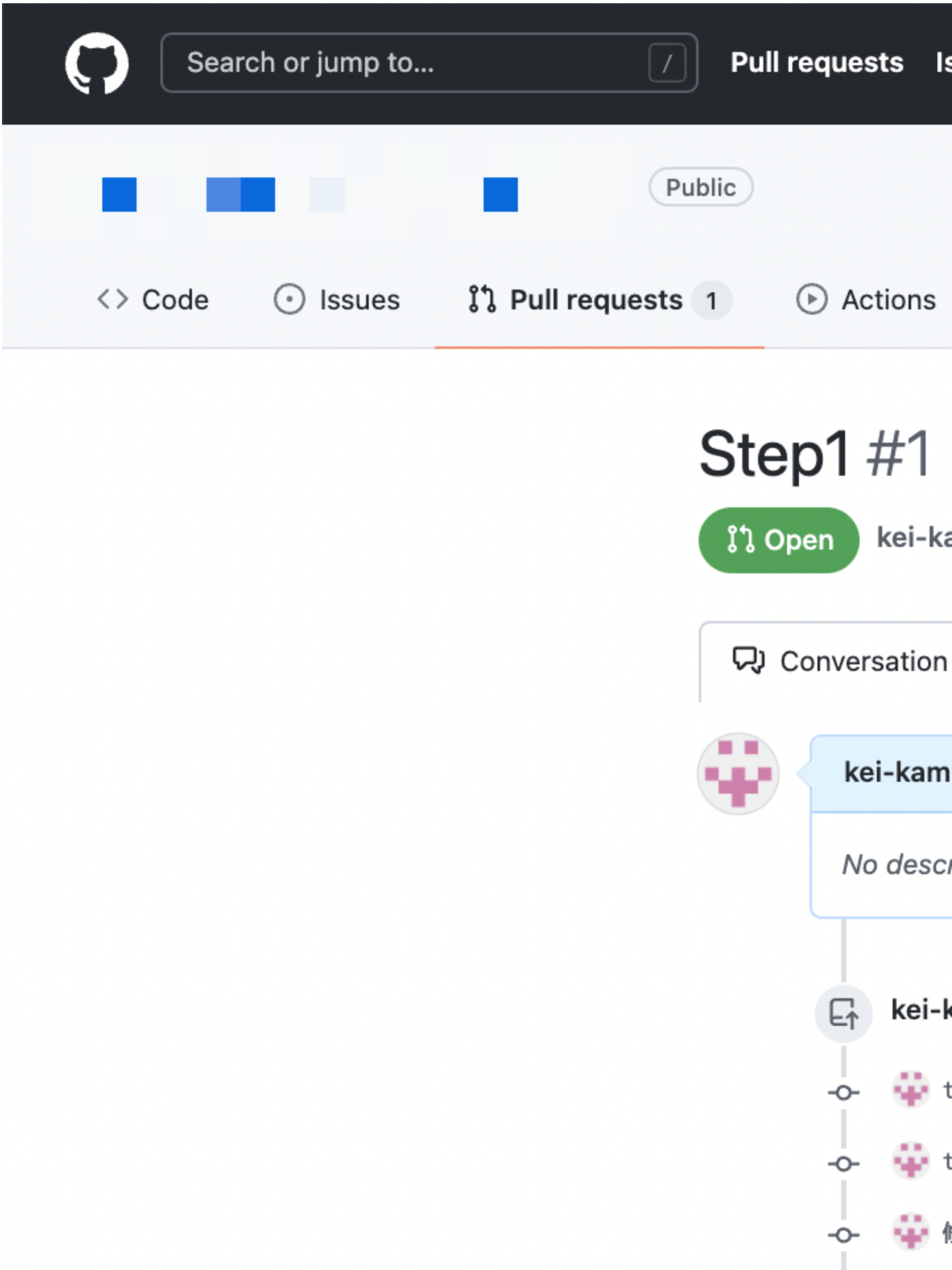
When all items have been ☑'d, proceed to “Submit assignment”.

2. submit assignment

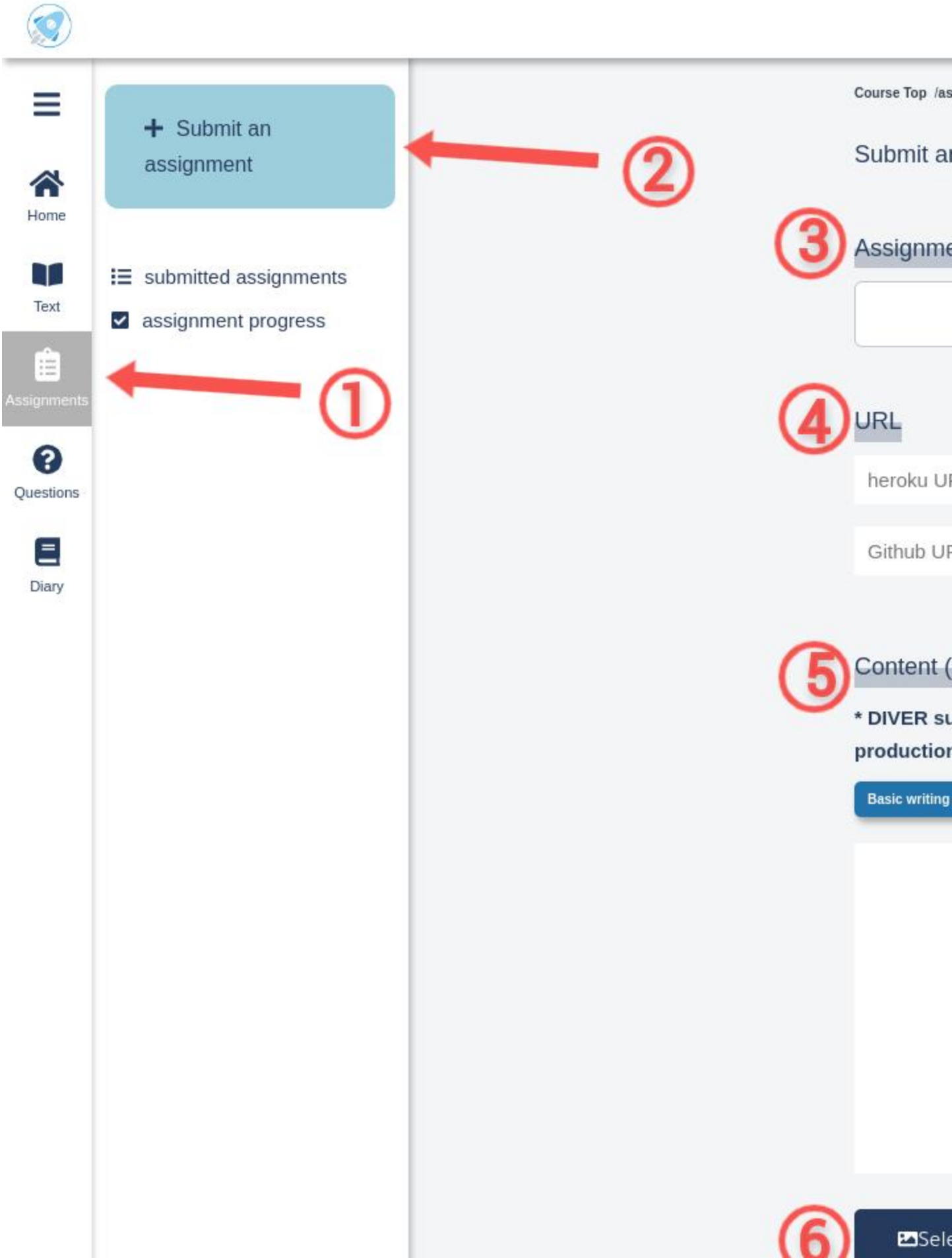
Create a pull request when all items are marked with ☑.
Follow the steps below to create a pull request.



Note that assignment must be submitted with the pull request submitted, and do not press the “Merge pull request” button that appears on the pull request.


Search or jump to...
/
Pull requests
Public
Code
Issues
Pull requests 1
Actions
Step1 #1
Open
Conversation
kei-kam

Once you have created your pull request, please follow these steps to submit assignment.


Home
Text
Assignments
Questions
Diary
+ Submit an assignment
submitted assignments
assignment progress
1
2
Course Top /as
Submit a
3
Assignme
4
URL
heroku U
Github U
5
Content (
* DIVER su
productio
Basic writing
6
Sele

①. Click on “assignment” in the sidebar.
ii. Click on “Submit assignment”.
3). Select assignment to submit.
④. Fill in the URL of the GitHub repository you created in this assignment (*Fill in the Heroku or AWS URL in assignment where you are required to deploy to the production environment).
⑤. Provide any remarks or other information you wish to convey.
⑥. For assignment, which requires submission of images, attach images
⑦. Click “Submit” and submit assignment

If you receive comments back from the mentor requesting revisions, correct the areas pointed out.
When the corrections are complete, push to the `step1 branch` again, confirm that all items are ☑, and submit another request for review from the comments section of assignment where you submitted the request.
When the mentor passes the step 1, click the “Merge pull request” button to merge the files.

table of contents

Manyo employee training assignment step 1

◀ Previous
Next ▶

- 1.Handling of assignment
- 2.About the license
- 3.Goal of assignment
- 4.Important: Flow from development to submission of assignment
- 5.Contents of step 1
- 6.Create a repository on GitHub
- 7.Clone the Rails application for the assignment
- 8.Create a test
- 9.Configure settings to prevent unnecessary files from being generated.
- 10.Implement CRUD functionality
- 11.Development Requirements
- 12.Screen Transition Requirements
- 13.Screen Design Requirements
- 14.Functional Requirements
- 15.Create a Model Spec
- 16.Create a System Spec
- 17.Create test data using FactoryBot
- 18.Verify
- 19.System Spec debugging
- 20.Deploy to Heroku

- 21.Link GitHub and Heroku
- 22.Pass requirements
- 23.Receive an evaluation of your assignment.

- Send feedback

Recommended version for assignments

Please submit your assignment in the following version. You do not need to consider languages not included in this assignment.

Ruby 3.X
Ruby on Rails 7.X